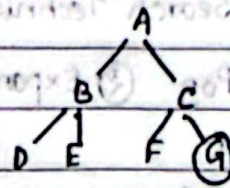


* Search BFS/DFS (Uninformed Search)

→ Goal state is reachable or not?

→ What is the path?



Parent List. →

X	A	A	B	E	C	C
---	---	---	---	---	---	---

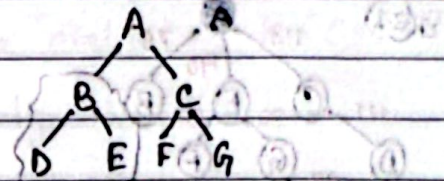
Path is the Sequence of A B C D E F G

Action.

Problem of BFS/DFS

→ Why unnecessary node visit?

Cannot solve this problem!



* That's why they are called ~~Blind~~

Blind Search or Uninformed Search

Not necessary

✓(can) → They can differentiate between Goal State and Non Goal State

X(cannot) → They can not " " Two or more Non-Goal States (which one more promising)

* Informed Search

→ Has the info of Heuristic → Helps pruning out less promising nodes.

→ An estimation of cheapest path cost from any node, 'n' to Goal node!

→ GBFS (Greedy Best First Search)

→ A* Search

→ Hill Climbing

⊛ We will judge Algorithm based on four categories

1. Completeness

2. Optimality

3. Space Complexity

4. Time Complexity

- Always return a solution

- Solution can be of any type, best, worst, average.

- Best solution

- Cheapest cost solution

- Total no. of nodes that are stored in worst case.

- Total no. of nodes that are visited in worst case.

How can we solve a problem? How can we make intelligent decisions? How can we solve a problem intelligently?

SS4320

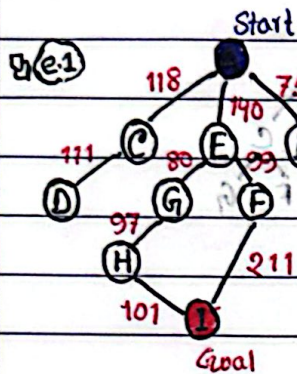
Date: .../.../...

Every Search Techniques follow 3 steps. 1. Pop 2. Expand 3. Goal Test

Queue, Stack, Priority Queue

Graph Search → Keeping the track of expanded or visited list.

Tree Search → Not keeping the track of expanded or visited node list.



State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

GBFS

$h(n)$ → heuristic fun^c

$f(n)$ → Evaluation fun^c

For GBFS,

$$f(n) = h(n)$$

Parent List:	*	A	A	A	E	E	F
	A	B	C	D	E	F	G

Simulation of e1. using

Tree search version of GBFS: .E

X	E ₂₅₃	C ₃₂₉	B ₃₇₄	G ₁₉₃	F ₁₇₈	P(I) = F; P(F) = E
						P(E) = A; P(A) = *

→ A₃₆₆

X	E ₂₅₃	C ₃₂₉	B ₃₇₄	G ₁₉₃	I ₀
---	------------------	------------------	------------------	------------------	----------------

X	A ₃₆₆	C ₃₂₉	E ₂₅₃	B ₃₇₄
---	------------------	------------------	------------------	------------------

✓	I ₀	C ₃₂₉	B ₃₇₄	G ₁₉₃	F ₁₇₈
---	----------------	------------------	------------------	------------------	------------------

∴ Path: A → E → F → I
140 + 99 + 211

∴ Total Path cost = 450

* Quality of GBFS:

1. Completeness	2. Optimality	3. Space Complexity	4. Time Complexity
- Not Complete (Tree search version)	- Not optimal - Sub-Optimal	- $O(b^m)$	- $O(b^m)$
- But Graph Search version is Complete.			$b \rightarrow$ Branching factor $m \rightarrow$ maximum path length

⇒ To make GBFS algorithm optimal

→ Should make some changes in the evaluation fun^c(f(n)),
which leads us to A* Search algorithm.

→ Uniform Cost Search (Kind of Dijkstra)

* A* Search

A* → UCS + GBFS

↳ Evaluation fun^c, $f(n) = h(n) + g(n)$

↳ Heuristic

↳ Actual Cost

- Goal node is
Consistent

- Starting node is
Consistent

Q Simulation of e1. using

Tree search version of A* Search

Parent list:	*	A	A	A				
	A	B	C	D	E	F	G	H

→ A₃₆₆

x A₃₆₆ C₄₄₇ E₃₉₃ B₄₄₉

x E₃₉₃ C₄₄₇ B₄₄₉ G₄₁₃ F₄₁₇

x G₄₁₃ C₄₄₇ B₄₄₉ F₄₁₇ H₄₁₅

x H₄₁₅ C₄₄₇ B₄₄₉ F₄₁₇ I₄₁₈

$$f(A) = g(A) + h(A) = 0 + 366 = 366$$

$$f(C) = 118 + 329 = 447$$

$$f(E) = 140 + 253 = 393$$

$$f(B) = 75 + 374 = 449$$

$$f(G) = 140 + 80 + 193 = 413$$

$$f(F) = 140 + 99 + 178 = 417$$

$$f(H) = 140 + 80 + 97 + 98 = 415$$

$$f(I) = 140 + 80 + 97 + 101 + 0 = 418$$

× F_{427} | C_{447} B_{449} L_{418}

$$F \rightarrow I: f(I) = 140 + 99 + 211 + 0 = 450$$

✓ L_{418} | C_{447} B_{449}

[As we already have L_{418} with less cost than this L_{450} , So we do not need to push this more costing L_{450} to the priority queue.]

$$P(I) = H, P(H) = G, P(G) = E, P(E) = A$$

$$P(A) = *$$

∴ Path: $A \rightarrow E \rightarrow G \rightarrow H$

$$140 + 80 + 99 + 0$$

$$\text{Total Cost} = 418$$

* Admissibility

- Estimated cost of going from one node to goal node must be less or equal than actual (lowest) cost.

- For every node:

$$\text{if } \{h(n) \leq c(n)\} \rightarrow \text{Admissible}$$

* Consistency

$$h(n) \leq c(n, n') + h(n')$$

$n \rightarrow$ current node

$n' \rightarrow$ Successor node

$c() \rightarrow$ cost func

Ex: $\overset{4}{(A)} \xrightarrow{1} \overset{1}{(B)} \xrightarrow{3} \overset{0}{(G)}$

$$h(A) \leq c(A, B) + h(B)$$

$$h(B) \leq c(B, G) + h(G)$$

$$h(A) \leq c(A)$$

$$\Rightarrow 4 \leq 1 + 1$$

$$\Rightarrow 1 \leq 3 + 0 \checkmark$$

$$\Rightarrow 4 \leq 4 \checkmark$$

$$\Rightarrow 4 \leq 2 \text{ X}$$

$$h(B) \leq c(B)$$

$$\Rightarrow 1 \leq 3 \checkmark$$

$$h(G) \leq c(G)$$

$$\Rightarrow 0 \leq 0 \checkmark$$

} Admissible

Not consistent

∴ Inconsistent but admissible heuristic.

A* Search to be

1. Complete

↳ Graph version → Need to be admissible

↳ Tree version → Need to be both admissible and consistent

⊛ All consistent heuristics are admissible. However, not all admissible heuristics are consistent.

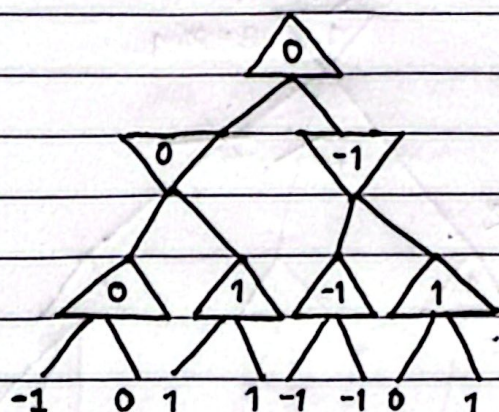
Adversarial Search

↳ Games with an opponent (Ex: Tik-tak-toe, Chess)

Minimax Algorithm

↳ My agent will always be at Max part.

↳ Traverse like BFS DFS



Max
 $d=2$

win = 1

Draw = 0

Min
 $d=1$

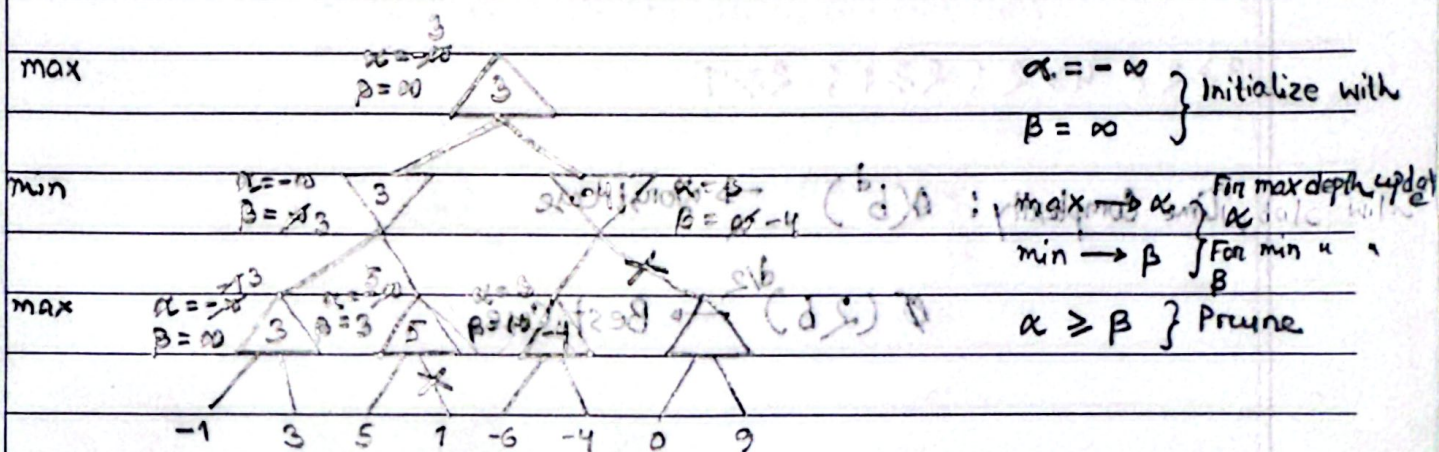
Lose = -1

$d = \text{Depth}$

Max
 $d=0$

Alpha-Beta Pruning

↳ One of the optimizations of Minimax Algorithm

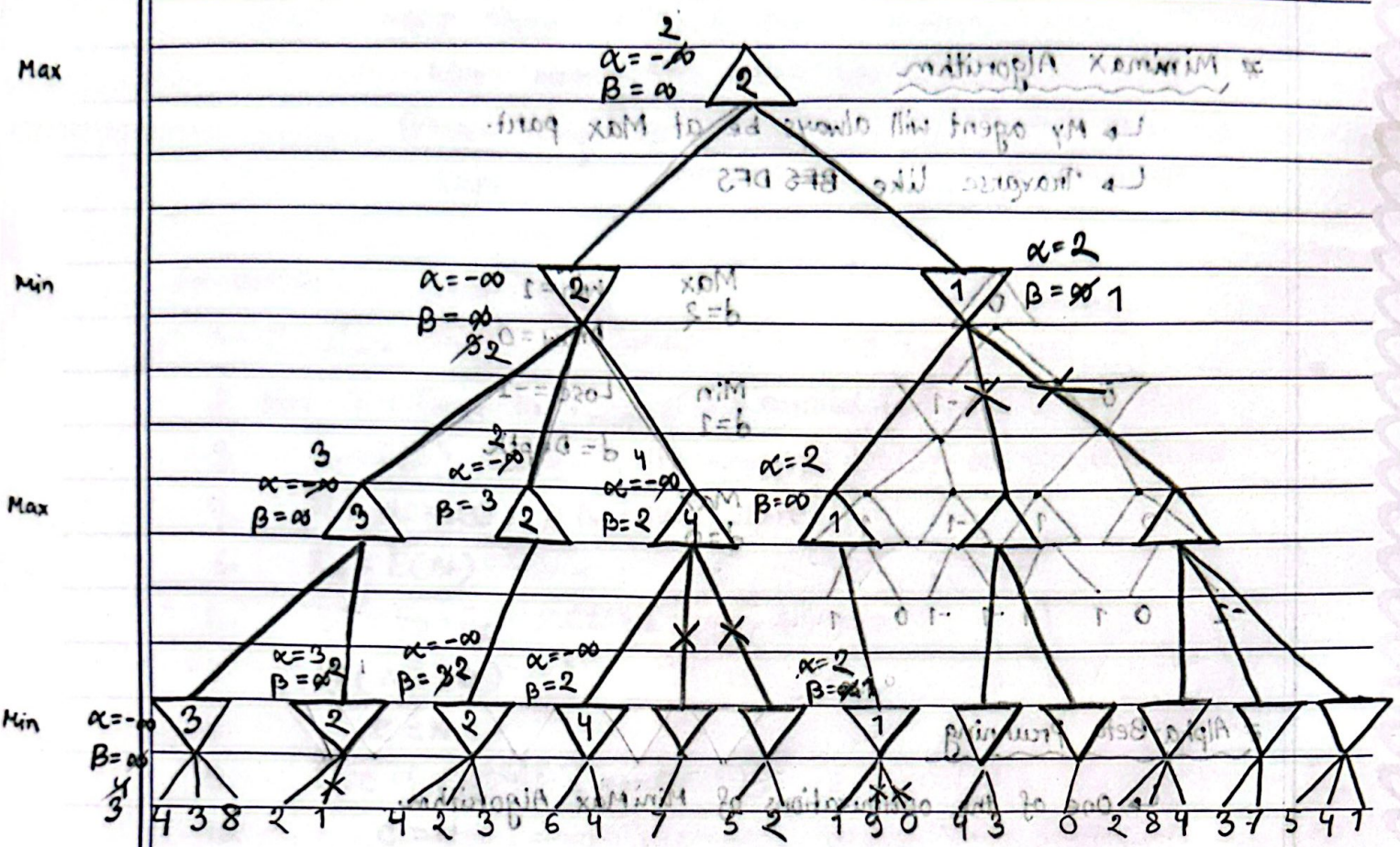


$5 > 3$; $3 > -4$

Date:/...../.....

Simulation Practice (α - β pruning):

to games with an opponent (for jk-jk-tas, chess)



$$3 \geq 2; 4 \geq 2; 2 \geq 1; 2 \geq 1$$

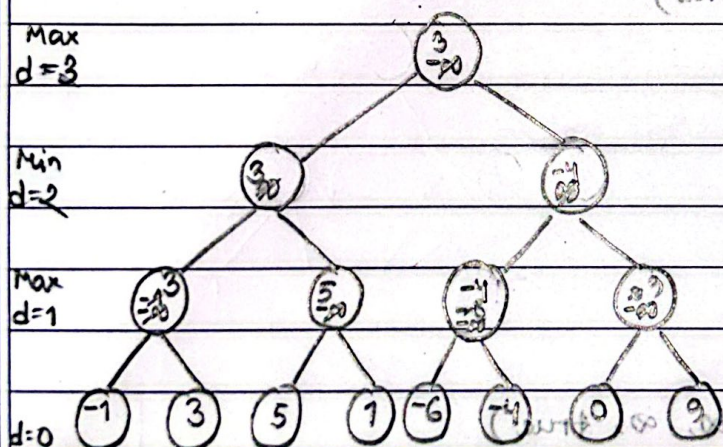
Time Complexity: $O(b^d)$ $\rightarrow$ worst case

$\Theta(2b)^{d/2} \rightarrow \text{Best Case.}$

Minimax Pseudocode:

1. function minimax(position, depth, maximizingPlayer)
2. if depth == 0 or game over in position
3. return static evaluation of position
- 4.
5. if maximizingPlayer
6. maxEval = $-\infty$
7. for each child of position
8. (eval = minimax(child, depth-1, false))
9. maxEval = max(maxEval, eval)
10. return maxEval
- 11.
12. else
13. minEval = ∞
14. for each child of position
15. eval = minimax(child, depth-1, true)
16. minEval = min(minEval, eval)
17. return minEval

Simulation:



Answer = 3

Pseudocode (α - β pruning):

```

1. function minimax(position, depth, alpha, beta, maximizingPlayer)
2.   if depth == 0 or game over in position
3.     return static evaluation of position
4.
5.   if maximizingPlayer
6.     maxEval =  $-\infty$ 
7.     for each child of position
8.       eval = minimax(child, depth-1, alpha, beta, false)
9.       maxEval = max(maxEval, eval)
10.      alpha = max(alpha, eval)
11.      if beta <= alpha
12.        break
13.     return maxEval
14.
15.   else
16.     minEval =  $\infty$ 
17.     for each child of position
18.       eval = minimax(child, depth-1, alpha, beta, true)
19.       minEval = min(minEval, eval)
20.       beta = min(beta, eval)
21.       if beta <= alpha
22.         break
23.     return minEval

```

// Initial call

minimax(currentPosition, 3, $-\infty$, ∞ , true)

↳ Not concerned with Path

↳ Solution itself matters

↳ Not connected with perfect solution

↳ Less Memory (No need to store the statespace)

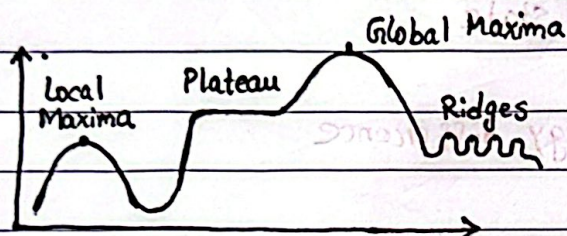
↳ Less time (No backtracking required)

↳ Applicable for large size graph.

⊗ Hill Climb Search

Step 1: Find the best child

Step 2: Check which one is better the parent or the children



Constrain: Can only move forward if and only if the child is better than the parent.

Drawbacks:

1. Can get stuck on local Maxima (/Minima)

2. " " " " Plateaus

3. " " " " Ridges

function Hill-Climbing(problem) # returns a state that is a local maxima

inputs: problem # a problem

local variables: current # a node
neighbor # a node

current ← Make-Node([Initial-State [problem]])

loop do

 neighbor ← a highest-valued successor of current

 if Value[neighbor] > Value[current]

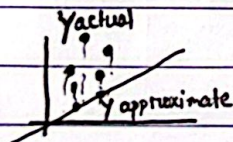
 then return State[current]

 current ← neighbor

Solutions:

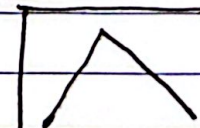
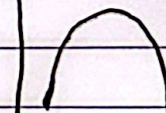
1. Random restart

2. Problem reformulation



One fun can be $\sum (Y_{act} - Y_{app})^2 \rightarrow y = x^2$

Another " " $\sum |Y_{act} - Y_{app}| \rightarrow y = |x|$



3. Simulated Annealing

↳ Uses control Parameter T (Temperature)

→ If Temp. is higher than a certain point then position keeps moving (even if it goes downhill)

⊙ When Temp. is lower than a certain point then searching stops.

Pseudocode:

1. $C = C_{init}$ // Here C = Current State and C_{init} = Initial State

2. For $T = T_{max}$ to T_{min} // T = control temp. before annealing.

3. $E_c = E(C)$ // E is the energy, utility or goodness value

4. $N = \text{Next}(C)$ // N = next state

5. $E_N = E(N)$

6. $\Delta E = E_N - E_c$ // ΔE = Energy difference

7. If $(\Delta E > 0)$

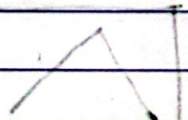
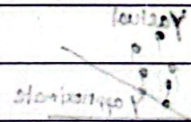
8. $C = N$

9. Else if $(e^{\Delta E/T} > \text{rand}(0,1))$ // Suppose, $\Delta E = -1$, $T_{max} = 100$ and $T_{min} = 2$

10. $C = N$ // $e^{\Delta E/T} = 0.99$ for $T_{max} = 100$

11. End // $e^{\Delta E/T} = 0.60$ for $T_{min} = 2$

1. Random restart
2. Problem reformulation

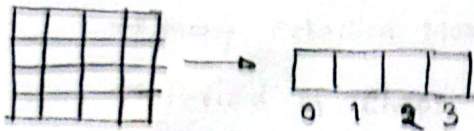


Genetic Algorithm

- * Population $\rightarrow$ Chromosomes
- * Selection (fitness function)
- * Crossover
- * Mutation

Q. 4-Queen

* Population



* Selection

fitness function $\rightarrow$ Minimizing No. of attacking pair Or Maximizing No. of Non-attacking pair

$$\begin{aligned} \text{Total number of pair} &= {}^nC_2 \\ &= {}^4C_2 \end{aligned}$$

$$f(x) = x^2 - 5x + 3 = 0$$

$$x = \{-15, +15\}$$

find a value where $f(x) = 0$

[Soln: Use binary representation]

$$2^4 = 16 + 1 \quad \uparrow \text{ For sign }$$

Algorithm 5: Genetic Algorithm	
1. <u>Input</u> : Population size, fitness function, mutation rate, crossover rate, maximum generation	1. Population Creation
2. <u>Output</u> : Best solution found.	2. Fitness calculation
3. <u>Begin</u>	3. Parent Selection
4. Initialize population with random candidates.	4. Crossover (child creation)
5. Evaluate the fitness of each candidate	5. Mutation
6. <u>while</u> termination condition not met <u>do</u>	6. Select individual for next gen.
7. Select parents from the current population	7. Goal Test
8. Perform crossover on parents to create new offspring	Yes → Terminate
9. Perform mutation on offspring	No → Loop (to 3 rd step)
10. Evaluate the fitness of new offspring	
11. Select individuals for next generation	
12. best solution in new generation > best solution so far	
13. Update best solution found	
14. <u>end while</u>	
15. <u>return</u> best solution found	
16. <u>end</u>	

* N-Queen Problem:

6				Q		
5						2
4	Q		Q			
3		Q		Q		
2						
1					2	Q

1st 2nd 3rd 4th 5th 6th

problem encoding

1. Population Creation:

Chromosome $\rightarrow C_1 \rightarrow 424631$

$C_2 \rightarrow 334315$

$C_3 \rightarrow 422351$

$C_4 \rightarrow 145231$

2. Fitness Calculation:

$C_1 \rightarrow 424631$

$clash = 1+1$

$Fitness(C_1) = \text{No. of max possible class} - \text{No. of clash}$

$$= {}^6C_2 - 2$$

$$= 15 - 2$$

$$= 13$$

C_2

$$\text{clash} = {}^3C_2 + 1 + 1 + 1 = 6$$

$$\text{Fitness}(C_2) = 6 - 6 = 0$$

C_3

$$\text{Fitness}(C_3) = 10$$

$C_4 \rightarrow 145231$

$$\text{Fitness}(C_4) = 11$$

3. Parent Selection:

- Truncation Selection
- Tournament
- Rank

Truncation Selection:

↳ best x% parent.

lets select C_1, C_3, C_4
(75%)

4. Crossover:

Single-Point

Randomly Select an index

$C_1 \rightarrow [4, 2, 4, 6, 3, 1]$
 $C_3 \rightarrow [4, 2, 2, 3, 5, 1]$

$C_5 \rightarrow [4, 2, 4, 3, 5, 1]$
 $C_6 \rightarrow [4, 2, 2, 6, 3, 1]$

1. Single-Point Crossover

2. two Point

$C_3 \rightarrow [4, 2, 2, 3, 5, 1]$

$C_4 \rightarrow [1, 4, 5, 2, 3, 1]$

$C_7 \rightarrow [4, 2, 2, 3, 3, 1]$

$C_8 \rightarrow [1, 4, 5, 2, 5, 1]$

Two-Point Crossover

$d = 1 + 1 + 1 + 1 = 4$
 $d = 4/10 = 0.4$
 4 2 2 3 5 1 1 4 2 3 3 1 C_7
 1 4 5 2 3 1 4 2 5 2 5 1 C_{10}

5. Mutation :

↳ x% Rate

let 10% → 0.1

if (random number(0,1) < 0.1)

Mutation

else

Skip

Randomly select index.

$C_5 \rightarrow 4 \ 2 \ 6 \ 3 \ 5 \ 1$

Then change the gene randomly (in range)

$C_6 \rightarrow$

6. Select Individual for next generation:

next gent: $C_5 \ C_6 \ C_7 \ C_8$
10 12 9 7

Elitism: - Next $\rightarrow C_1 \ C_5 \ C_6 \ C_7$
13